

组合数据类型概述



元组 (Tuple)

1. 元组的定义
2. 元组的运算
3. 元组的打包和解包
4. 星号表达式
5. 元组与列表的比较
6. 元组与列表的转换
7. 序列类型的共性

元组 (Tuple)

元组的定义

- **定义**：元组也是有序的数据序列，与列表的区别在于元组是不可变类型，创建之后不能添加、删除或修改元素
- **创建方式**：用圆括号把元素括起来，元素之间用逗号分隔
- **语法**：元组名 = (元素1, 元素2, ..., 元素n)

元组 (Tuple)

■ 元组的创建与访问

```
t1 = (35, 12, 98)                    # 用圆括号创建元组
t2 = ('张三', 18, True)               # 元素类型可以不同
print(t1)
print(t2)
print(type(t1))
print(len(t2))                       # 元素个数
print(t2[0])                         # 索引访问，与列表相同
print(t2[-1])
```

运行结果：

```
(35, 12, 98)
('张三', 18, True)
<class 'tuple'  >
3
张三
True
```

元组 (Tuple)

空元组与一个元素的元组

- 创建只有一个元素的元组时，元素后面必须加逗号

```
t1 = ()                          # 空元组
t2 = ('hello')                   # 只有圆括号，没有逗号
t3 = ('hello', )                 # 有逗号
print(type(t1))
print(t2, type(t2))              # 结果是字符串，不是元组
print(t3, type(t3))              # 结果是元组
print(len(t2), len(t3))


t4 = (100)                       # 圆括号被当作运算的括号
t5 = (100, )
print(type(t4), type(t5))
```

运行结果：

```
<class 'tuple'>
hello <class 'str'>
('hello',) <class 'tuple'>
5 1
<class 'int'> <class 'tuple'>
```

元组 (Tuple)

空元组与一个元素的元组

- 创建只有一个元素的元组时，元素后面必须加逗号
  - 圆括号在 Python 中还有改变运算顺序的用途。只有加上逗号，Python 才知道这是一个元组
  - ⚠ 方括号没有这个问题，所以列表不需要加逗号

写法	类型	长度
`()`	元组（空）	0
`('hello')`	字符串	5
`('hello', )`	元组	1
`['hello']`	列表	1

元组 (Tuple)

- **元组不能修改**

```
t2 = ('张三', 18, True)
t2[0] = '李四'                    # 尝试修改元素
```

报错：TypeError: 'tuple' object does not support item assignment

```
t2.append('男')                   # 尝试添加元素
```

报错：AttributeError: 'tuple' object has no attribute 'append'

元组 (Tuple)

元组的运算

- 凡是不修改元组本身的操作，元组都支持，用法与列表完全相同

运算	列表	元组
索引、切片	支持	支持
成员运算 `in`	支持	支持
拼接 `+`、重复 `*`	支持	支持
比较运算	支持	支持
遍历 `for`	支持	支持
`len()`	支持	支持
修改元素、增删元素	支持	**不支持**

元组 (Tuple)

元组的运算

- 凡是不修改元组本身的操作，元组都支持，用法与列表完全相同

拼接和重复得到的是新元组，原元组不变，这与“元组不可变”并不矛盾：变的不是原元组，而是产生了一个新元组

```
t1 = (35, 12, 98)
t2 = ('张三', 18)
print(t1 + t2)              # 拼接：生成一个新元组
print(t1 * 2)               # 重复：生成一个新元组
print(t1[1:])               # 切片：结果仍是元组
print(12 in t1)             # 成员运算
print(t1 < (35, 13))        # 比较运算：逐个元素比较
for x in t2:                # 遍历
    print(x, end=' ')
print()
print(t1)                   # 原元组没有改变
```

不是列表

(35, 12, 98) < (35, 13) 中第一对相等，第二对 12 < 13，结果为 True

元组 (Tuple)

元组的运算

- 凡是不修改元组本身的操作，元组都支持，用法与列表完全相同

```
t1 = (35, 12, 98)
t2 = ('张三', 18)
print(t1 + t2)              # 拼接：生成一个新元组
print(t1 * 2)               # 重复：生成一个新元组
print(t1[1:])               # 切片：结果仍是元组
print(12 in t1)             # 成员运算
print(t1 < (35, 13))        # 比较运算：逐个元素比较
for x in t2:                # 遍历
    print(x, end=' ')
print()
print(t1)                   # 原元组没有改变
```

运行结果：

```
(35, 12, 98, '张三', 18)
(35, 12, 98, 35, 12, 98)
(12, 98)
True
True
张三 18
(35, 12, 98)
```

元组 (Tuple)

元组的打包和解包

- **打包**：把多个值用逗号连接，赋值给一个变量，Python会自动把这些值组成一个元组。圆括号可以省略
- **解包**：把一个元组中的元素依次赋值给多个变量

```
student = '张三', 18, True          # 打包：不写圆括号也是元组
print(student, type(student))


name, age, is_male = student        # 解包：把 3 个元素分别赋值给 3 个变量
print(name)
print(age)
print(is_male)
```

运行结果：

('张三', 18, True) <class 'tuple’ >
张三
18
True

元组 (Tuple)

元组的打包和解包

- ⚠️ 解包时变量个数必须与元素个数相同，否则报错：

```
student = '张三', 18, True

name, age = student                          # 变量少了
# ValueError: too many values to unpack (expected 2)

name, age, is_male, height = student     # 变量多了
# ValueError: not enough values to unpack (expected 4, got 3)
```

元组 (Tuple)

■ 星号表达式

- **问题**：解包时要求变量个数与元素个数相同，如果元素很多、只关心其中几个怎么办？
- **解决办法**：在某个变量前加 *，让它接收剩下的全部元素

```
scores = 95, 83, 92, 78, 66

first, *rest = scores              # first 接收第一个元素，rest 接收剩下的全部
print(first, rest)

*front, last = scores              # last 接收最后一个元素，front 接收前面的全部
print(front, last)

first, *middle, last = scores      # middle 接收中间的全部
print(first, middle, last)

print(type(rest))                  # 星号变量接收到的是列表，不是元组
```

运行结果：

```
95 [83, 92, 78, 66]
[95, 83, 92, 78] 66
95 [83, 92, 78] 66
<class 'list'>
```

元组 (Tuple)

星号表达式

- **规则**：
  - 星号变量接收到的结果是列表，即使被解包的是元组
  - 一次解包中，星号变量只能出现一次，否则 Python 无法确定怎么分配元素

```
a, *b, *c = scores
```

报错 SyntaxError: multiple starred expressions in assignment

  - 其他变量优先分配，星号变量可能一个元素也分不到，此时得到空列表：

```
scores = 95, 83
a, b, *c = scores
print(a, b, c)      # 95 83 []
```

元组 (Tuple)

星号表达式

- ⚠️ 解包适用于所有序列类型

```
a, *b = 'hello'          # 字符串
print(a, b)              # h ['e', 'l', 'l', 'o']
a, *b = [1, 2, 3]        # 列表
print(a, b)              # 1 [2, 3]
a, *b = range(4)         # range
print(a, b)              # 0 [1, 2, 3]
```

元组 (Tuple)

元组与列表的比较

- **问题**：元组能做的事列表都能做，而且列表还能修改，为什么还需要元组？
  - **理由一**：元组不可变，数据更安全
    - 用元组保存，任何试图修改的代码都会立刻报错，避免数据被误改
  - **理由二**：创建速度更快，占用空间更小

```
import timeit
print('创建列表：', round(timeit.timeit('[1, 2, 3, 4, 5, 6, 7, 8, 9]',
number=1000000), 3), '秒')
print('创建元组：', round(timeit.timeit('(1, 2, 3, 4, 5, 6, 7, 8, 9)',
number=1000000), 3), '秒')
```

    - 创建同样 9 个元素，重复一百万次，元组的创建速度快了约 4-6 倍 (因计算机性能不同，时间会有差异)
    - 原因是列表要为将来的增删预留空间、做额外准备，元组创建后不会变化，不需要这些开销

元组 (Tuple)

元组与列表的转换

函数	作用
`tuple(列表)`	把列表转换成元组
`list(元组)`	把元组转换成列表

```
scores = [95, 83, 92]
t = tuple(scores)          # 列表 → 元组
print(t, type(t))

student = ('张三', 18, True)
ls = list(student)         # 元组 → 列表
print(ls, type(ls))
```

运行结果：

(95, 83, 92) <class 'tuple’ >
['张三', 18, True] <class 'list’ >

元组 (Tuple)

序列类型的共性

- 字符串、列表、元组都是序列类型，元素有先后顺序，因此支持同一套操作

操作	说明	字符串	列表	元组
`len(s)`	元素个数	支持	支持	支持
`s[i]`	索引访问	支持	支持	支持
`s[i:j:k]`	切片	支持	支持	支持
`x in s`	成员运算	支持	支持	支持
`s + t`	拼接	支持	支持	支持
`s * n`	重复	支持	支持	支持
`for x in s`	遍历	支持	支持	支持
比较运算	逐个元素比较	支持	支持	支持
解包	`a, *b = s`	支持	支持	支持
`s.index(x)`	查找位置	支持	支持	支持
`s.count(x)`	统计次数	支持	支持	支持
`s[i] = x`	修改元素	**不支持**	支持	**不支持**
`append`、`remove`等	增删元素	**不支持**	支持	**不支持**

元组 (Tuple)

序列类型的共性

- 字符串、列表、元组都是序列类型，元素有先后顺序，因此支持同一套操作

```
s = 'python'
ls = [95, 83, 92]
t = ('张三', 18, True)

print(len(s), len(ls), len(t))          # 长度
print(s[0], ls[0], t[0])                # 索引
print(s[1:3], ls[1:3], t[1:3])          # 切片
print('y' in s, 83 in ls, 18 in t)      # 成员运算
print(ls.count(95), t.count(18), s.count('p'))    # 统计次数
print(ls.index(83), t.index(18), s.index('t'))    # 查找位置
```

运行结果：

```
6 3 3
p 95 张三
yt [83, 92] (18, True)
True True True
1 1 1
1 1 2
```

元组 (Tuple)

练习题

第 1 题　写出下面代码的运行结果

```python
t = (10, 20, 30, 20, 40)
print(t[1:4])
print(t[::-1])
print(len(t), t.count(20), t.index(20))
print(t + (50, ))
print(t)
```

元组 (Tuple)

练习题

索引	0	1	2	3	4
元素	10	20	30	20	40

表达式	求值过程	结果
`t[1:4]`	取索引 1、2、3，不含索引 4；切片结果**仍是元组**	`(20, 30, 20)`
`t[::-1]`	步长 -1，从右向左取全部元素	`(40, 20, 30, 20, 10)`
`len(t)`	元素个数	`5`
`t.count(20)`	20 出现了 2 次（索引 1 和索引 3）	`2`
`t.index(20)`	20 **第一次**出现的位置	`1`
`t + (50, )`	拼接生成**新元组**，注意 `(50, )` 的逗号	`(10, 20, 30, 20, 40, 50)`
`print(t)`	拼接不修改原元组	`(10, 20, 30, 20, 40)`

元组 (Tuple)

练习题

第 2 题　写出下面代码的运行结果，并说明原因

```python
a = ('python')
b = ('python', )
c = ()
print(type(a), len(a))
print(type(b), len(b))
print(type(c), len(c))
```

元组 (Tuple)

练习题

变量	写法	有无逗号	判定	长度的含义
a	`('python')`	无	圆括号被当作**运算的括号**，里面是字符串	字符串的字符个数：6
b	`('python',)`	有	逗号决定了这是元组	元组的元素个数：1
c	`()`	—	空元组，没有歧义	0

运行结果：

<class 'str'> 6
<class 'tuple'> 1
<class 'tuple'> 0



集合 (set)

1. 创建集合
2. 集合的特点
3. 集合元素的要求
4. 集合的成员运算与遍历
5. 集合的运算：交、并、差、对称差
6. 集合的比较运算
7. 集合的方法
8. 不可变集合 frozenset
9. 集合的应用场景

集合 (set)

创建集合

- **方式一**：使用大括号 {}
  - 集合名 = {元素1, 元素2, ..., 元素n}

```
s1 = {35, 12, 99}                    # 用大括号创建集合
s2 = {'Python', 'Java', 'C++'}       # 元素也可以是字符串
s3 = {100, 12.3, 'Python', (3, 5)}   # 元素类型可以不同，也可以是元组
print(s1)
print(s2)
print(s3)
print(type(s1))
```

运行结果：

```
{35, 99, 12}
{'C++', 'Java', 'Python’}
{(3, 5), 100, 12.3, 'Python'}
<class 'set'>
```

⚠ 集合的元素类型可以不同，但不能是列表

集合 (set)

创建集合

- **方式二**：把列表、元组、字符串、range 等序列转换成集合
  - 集合变量名 = set(序列)

```
s3 = set([95, 83, 92, 95])          # 用 set() 把列表转换成集合
s4 = set((95, 83, 92, 95))          # 元组同样可以转换
s5 = set('hello')                   # 把字符串转换成集合
print(s3)
print(s4)
print(s5)
```

转换过程中会自动去掉重复元素

运行结果：

```
{83, 92, 95}
{83, 92, 95}
{'o', 'h', 'l', 'e'}
```

集合 (set)

集合元素的要求

- 集合的元素必须是不可变类型

可以作为集合元素	不能作为集合元素
整数、浮点数、布尔值	列表
字符串	集合
元组（元素也必须不可变）	字典

```
s1 = {[95, 83]}          # 列表是可变类型
# TypeError: unhashable type: 'list'

s2 = {(1, [2, 3])}       # 元组本身不可变，但里面套了列表
# TypeError: unhashable type: 'list'

s3 = {{1, 2}}            # 集合本身也是可变类型
# TypeError: unhashable type: 'set'
```

集合 (set)

创建空集合

- 使用 **set()** 而非 {}

```
s7 = set()                                    # 创建空集合
print(s7, type(s7))
s8 = {}                                       # 注意：这是空字典，不是空集合
print(s8, type(s8))
```

运行结果：
set() <class 'set' >
{} <class 'dict'>

⚠ 空集合必须用 set() 创建。{} 表示的是空字典，因为大括号最早是给字典用的

集合 (set)

■ 数学中的集合与 Python 中的集合

- Python 中的集合与中学数学中的集合概念一致，具有三个基本特性：

特性	含义	在 Python 中的表现
无序性	元素没有先后顺序	不支持索引，输出顺序与写入顺序可能不同
互异性	元素互不相同	重复的元素只保留一个
确定性	任何元素都能明确判断是否属于该集合	用 `in` 判断，结果是 True 或 False

```
s1 = {3, 1, 2}
print(s1)                          # 输出顺序与写入顺序可能不同：无序性

s2 = {1, 2, 2, 3, 3, 3}
print(s2)                          # 重复的元素只保留一个：互异性

print(2 in s2, 5 in s2)            # 任何元素都能明确判断是否属于集合：确定性
print(len(s2))
```

运行结果：

```
{1, 2, 3}
{1, 2, 3}
True False
3
```

集合 (set)

集合不支持索引和切片

- 集合是无序的，元素没有位置编号

```
s = {35, 12, 99}
print(s[0])
```

报错 TypeError: 'set' object is not subscriptable

集合 (set)

集合的成员运算与遍历

- 成员运算 in、not in 和遍历的写法与列表、元组完全相同
- 集合无序，遍历顺序不确定，需要按顺序处理数据时不要用集合

```
languages = {'Python', 'Java', 'C++', 'Go'}

print('Python' in languages)          # 成员运算：判断元素是否在集合中
print('Swift' not in languages)

for language in languages:            # 遍历集合中的每个元素
    print(language, end=' ')
print()
print(len(languages))                 # 元素个数
```

运行结果：

```
True
True
Python Java Go C++ (遍历顺序由 Python 内部决定)
4
```

集合 (set)

集合的运算：交集、并集、差集、对称差

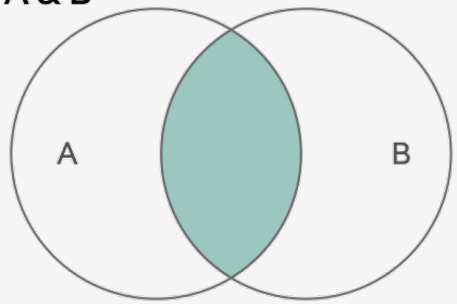
运算	运算符	含义
交集	`A & B`	同时属于 A 和 B 的元素
并集	`A | B`	属于 A 或属于 B 的元素
差集	`A - B`	属于 A 但不属于 B 的元素
对称差	`A ^ B`	只属于其中一个集合的元素（即并集去掉交集）

集合 (set)

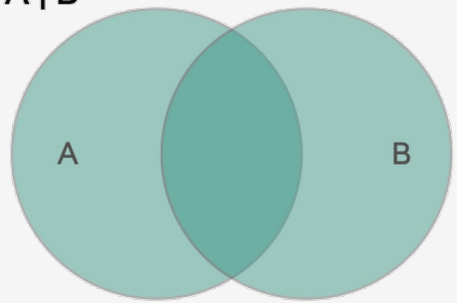
集合的运算：交集、并集、差集、对称差

- 韦恩图（阴影部分表示运算结果）
  - 韦恩图：用图形表示集合关系的一种图示方法，由英国数学家约翰·韦恩（John Venn）提出

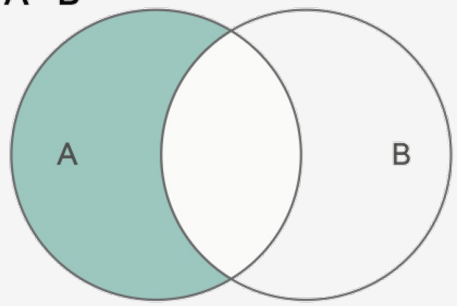
交集 A & B



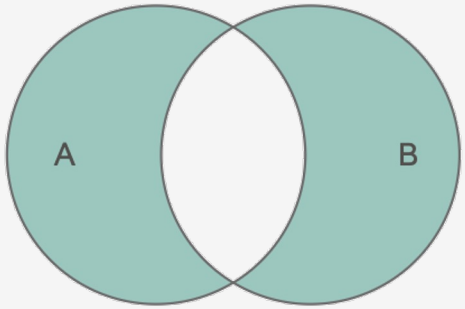
并集 A | B



差集 A - B



对称差 A ^ B



集合 (set)

■ 集合的运算：交集、并集、差集、对称差

```
set1 = {1, 2, 3, 4, 5, 6, 7}
set2 = {2, 4, 6, 8, 10}

print(set1 & set2)      # 交集：同时属于两个集合的元素
print(set1 | set2)      # 并集：属于任意一个集合的元素
print(set1 - set2)      # 差集：属于 set1 但不属于 set2 的元素
print(set2 - set1)      # 差集：属于 set2 但不属于 set1 的元素
print(set1 ^ set2)      # 对称差：只属于其中一个集合的元素
```

运行结果：

```
{2, 4, 6}
{1, 2, 3, 4, 5, 6, 7, 8, 10}
{1, 3, 5, 7}
{8, 10}
{1, 3, 5, 7, 8, 10}
```

⚠ 差集不满足交换律：set1 - set2 与 set2 - set1 结果不同

⚠ 集合的交集、并集、对称差都满足交换律

⚠ 对称差等于两个差集合并起来

⚠ 四种运算都生成新集合，原来的两个集合不变

集合 (set)

■ 集合的运算：交集、并集、差集、对称差

- 每种运算都有对应的方法
- ⚠️ 结果返回一个新的集合，原集合不变

运算	运算符写法	方法写法
交集	`set1 & set2`	`set1.intersection(set2)`
并集	`set1 | set2`	`set1.union(set2)`
差集	`set1 - set2`	`set1.difference(set2)`
对称差	`set1 ^ set2`	`set1.symmetric_difference(set2)`

集合 (set)

■ 集合运算的方法写法与复合赋值

- 每种运算都有对应的方法
- ⚠ 结果返回一个新的集合，原集合不变

```
set1 = {1, 2, 3, 4, 5, 6, 7}
set2 = {2, 4, 6, 8, 10}

print(set1.intersection(set2))          # 交集，等价于 set1 & set2
print(set1.union(set2))                 # 并集，等价于 set1 | set2
print(set1.difference(set2))            # 差集，等价于 set1 - set2
print(set1.symmetric_difference(set2))  # 对称差，等价于 set1 ^ set2
print(set1)                             # 原集合没有改变
```

运行结果：

```
{2, 4, 6}
{1, 2, 3, 4, 5, 6, 7, 8, 10}
{1, 3, 5, 7}
{1, 3, 5, 7, 8, 10}
{1, 2, 3, 4, 5, 6, 7}
```

集合 (set)

■ 集合运算的方法写法与复合赋值

- **复合赋值运算**：先运算，再把结果赋值回原变量

```
set1 = {1, 2, 3, 4, 5, 6, 7}
set2 = {2, 4, 6, 8, 10}

set1 &= set2          # 交集运算的结果直接更新到 set1，等价于 set1 = set1 & set2
print(set1)

set3 = {1, 2, 3}
set3 |= {4, 5}        # 并集运算的结果直接更新到 set3
print(set3)
set3 -= {1, 2}        # 差集运算的结果直接更新到 set3
print(set3)
```

运行结果：

{2, 4, 6}
{1, 2, 3, 4, 5}
{3, 4, 5}

集合 (set)

集合的比较运算

- 集合的比较不是比大小，而是判断两个集合之间的包含关系

运算符	含义	数学符号
`A == B`	A 与 B 相等（元素完全相同）	$A = B$
`A <= B`	A 是 B 的子集	$A \subseteq B$
`A < B`	A 是 B 的真子集	$A \subsetneq B$
`A >= B`	A 是 B 的超集	$A \supseteq B$
`A > B`	A 是 B 的真超集	$A \supsetneq B$

集合 (set)

集合的比较运算

- 集合的比较不是比大小，而是判断两个集合之间的包含关系

```
set1 = {1, 3, 5}
set2 = {1, 2, 3, 4, 5}
set3 = {5, 4, 3, 2, 1}

print(set2 == set3)           # 相等：元素完全相同，与顺序无关
print(set1 < set2)            # 真子集：set1 的元素都在 set2 中，且 set2 还有别的元素
print(set1 <= set2)           # 子集：set1 的元素都在 set2 中
print(set2 < set3)            # set2 与 set3 相等，不是真子集
print(set2 <= set3)           # 相等的集合互为子集
print(set2 > set1)            # 真超集
print(set1.issubset(set2))    # 子集，等价于 set1 <= set2
print(set2.issuperset(set1)) # 超集，等价于 set2 >= set1
```

运行结果：

```
True
True
True
False
True
True
True
True
```

集合 (set)

集合的方法

功能	方法	作用
添加元素	`add(x)`	添加一个元素 x，已存在则无效果
	`update(集合)`	一次添加多个元素
删除元素	`discard(x)`	删除元素 x，**不存在也不报错**
	`remove(x)`	删除元素 x，**不存在会报错**
	`pop()`	删除并返回**任意**一个元素
	`clear()`	清空集合
判断关系	`isdisjoint(集合)`	两个集合没有相同元素时返回 True

集合 (set)

■ 集合的方法

```
languages = {'Python', 'Java'}

languages.add('C++')                # 添加一个元素
languages.add('Python')             # 已存在的元素，添加无效果
print(languages)

languages.update({'Go', 'Rust'}) # 一次添加多个元素
print(languages)

languages.discard('Rust')           # 删除元素，不存在也不报错
languages.discard('Swift')
print(languages)

languages.remove('Go')              # 删除元素，不存在会报错
print(languages)

x = languages.pop()                 # 删除并返回任意一个元素
print(x, languages)

languages.clear()                   # 清空集合
print(languages)
```

运行结果：

{'Java', 'C++', 'Python'}
{'C++', 'Go', 'Python', 'Java', 'Rust'}
{'C++', 'Go', 'Python', 'Java'}
{'C++', 'Python', 'Java'}
C++ {'Python', 'Java'}
set()

集合 (set)

集合的方法

```
a = {'Python', 'Java'}
b = {'HTML', 'CSS'}
c = {'Java', 'Go'}
print(a.isdisjoint(b), a.isdisjoint(c))   # True False
```

a 与 b 没有共同元素，返回 True
a 与 c 都有 'Java'，返回 False

集合 (set)

不可变集合 frozenset

- frozenset: 不可变的集合，创建后不能添加或删除元素。它与 set 的关系，就像元组与列表的关系
- 创建方式：frozenset(序列)

```
fset1 = frozenset({1, 3, 5, 7})          # 由集合创建
fset2 = frozenset(range(1, 6))           # 由 range 创建
print(fset1)
print(fset2)
print(fset1 & fset2)                     # 交集
print(fset1 | fset2)                     # 并集
print(fset1 - fset2)                     # 差集
print(fset1 < fset2)                     # 比较运算
print(3 in fset1)                        # 成员运算
```

```
frozenset({1, 3, 5, 7})
frozenset({1, 2, 3, 4, 5})
```

⚠ frozenset 支持集合的所有运算（交、并、差、比较、成员运算），但**不支持** add、remove 等修改自身的方法

集合 (set)

不可变集合 frozenset

- frozenset: 不可变的集合，创建后不能添加或删除元素。它与 set 的关系，就像元组与列表的关系
- 创建方式：frozenset(序列)

```
fset1.add(9)
```

报错 AttributeError: 'frozenset' object has no attribute 'add'

集合 (set)

集合的应用场景

- 集合最常用于三种场景：去重、成员测试、关系运算
- **场景一**：去重

```
nums = [95, 83, 92, 95, 83, 78]
unique = set(nums)                    # 转成集合，自动去掉重复元素
print(unique)
print(len(unique))                    # 有几个不同的分数
nums = list(set(nums))                # 需要继续当列表用时，再转回列表
print(nums)
```

运行结果：

```
{83, 92, 78, 95}
4
[83, 92, 78, 95]
```

集合 (set)

集合的应用场景

- 集合最常用于三种场景：去重、成员测试、关系运算
- **场景二**：成员测试

```
courses = {'Python', 'Java', 'C++', 'Go'}    # 本学期开设的课程
choice = 'Rust'
if choice in courses:
    print(choice, '可以选')
else:
    print(choice, '本学期未开设')
```

数据量大、需要频繁判断“在不在”时，用集合比列表快得多
原因在于：列表判断 in 要从头到尾逐个比较，数据越多越慢；集合通过元素的值直接算出存放位置，一步就能找到

集合 (set)

集合的应用场景

- 集合最常用于三种场景：去重、成员测试、关系运算
- **场景三**：关系运算

```
class1 = {'张三', '李四', '王五', '赵六'}      # 选修 Python 的学生
class2 = {'李四', '赵六', '孙七'}              # 选修 Java 的学生
print(class1 & class2)                  # 两门都选的学生
print(class1 | class2)                  # 至少选了一门的学生
print(class1 - class2)                  # 只选了 Python 的学生
```

运行结果：

{'赵六', '李四' }
{'孙七', '张三', '李四', '赵六', '王五'}
{'王五', '张三' }

集合 (set)

练习题

第 1 题　写出下面代码的运行结果

```python
s = set([3, 1, 4, 1, 5, 9, 2, 6, 5])
print(len(s))
s.add(4)
s.add(7)
s.discard(1)
s.discard(8)
print(len(s))
print(1 in s, 7 in s)
```

集合 (set)

■ 练习题

代码	执行后的集合	元素个数	说明
`s = set([...])`	`{1, 2, 3, 4, 5, 6, 9}`	7	自动去重
`s.add(4)`	`{1, 2, 3, 4, 5, 6, 9}`	7	4 已存在，添加无效果
`s.add(7)`	`{1, 2, 3, 4, 5, 6, 7, 9}`	8	7 是新元素
`s.discard(1)`	`{2, 3, 4, 5, 6, 7, 9}`	7	删除 1
`s.discard(8)`	`{2, 3, 4, 5, 6, 7, 9}`	7	8 不存在，**不报错**，什么也不做

运行结果：

7
7
False True

集合 (set)

■ 练习题

第 2 题　已知 `A = {1, 2, 3, 4}`，`B = {3, 4, 5}`，写出下列表达式的结果

python

```
print(A & B)
print(A | B)
print(A - B)
print(B - A)
print(A ^ B)
print({3, 4} <= A, {3, 4} < B, A == B)
```

运行结果：

```
{3, 4}
{1, 2, 3, 4, 5}
{1, 2}
{5}
{1, 2, 5}
True True False
```

集合 (set)

练习题

第 3 题　两门选修课的报名名单如下（名单中有重复报名的情况）

```python
python_list = ['张三', '李四', '王五', '李四', '赵六']
java_list = ['李四', '赵六', '孙七', '孙七']
```

用集合完成：

1. 两门课实际各有多少名学生?
2. 哪些学生两门课都选了?
3. 哪些学生只选了其中一门?
4. 一共有多少名不同的学生报名?

集合 (set)

练习题

```
python_list = ['张三', '李四', '王五', '李四', '赵六']
java_list = ['李四', '赵六', '孙七', '孙七']

py = set(python_list)                    # 去重：李四重复报名
jv = set(java_list)                      # 去重：孙七重复报名
print(len(py), len(jv))                  # 问题 1
print(py & jv)                           # 问题 2：两门都选
print(py ^ jv)                           # 问题 3：只选一门
print(len(py | jv))                      # 问题 4：一共多少人
```

运行结果：

```
4 3
{'李四', '赵六'}
{'孙七', '张三', '王五'}
5
```